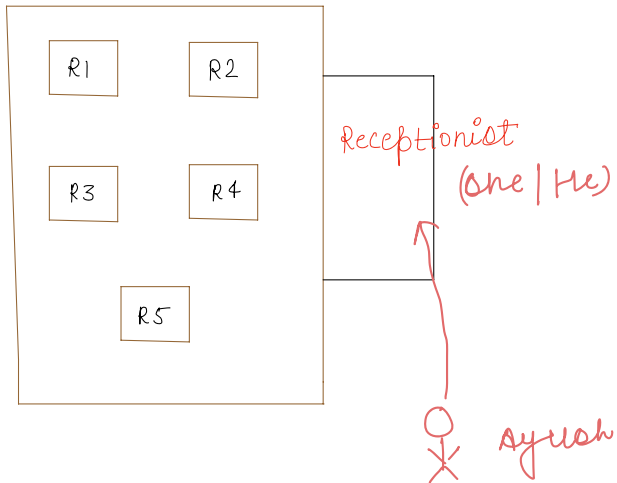


Agenda

- HashMap introduction
- HashSet introduction
- frequency of each el
- Count distinct elements
- Pair sum = k
- Count pair sum = k [H/w maybe]

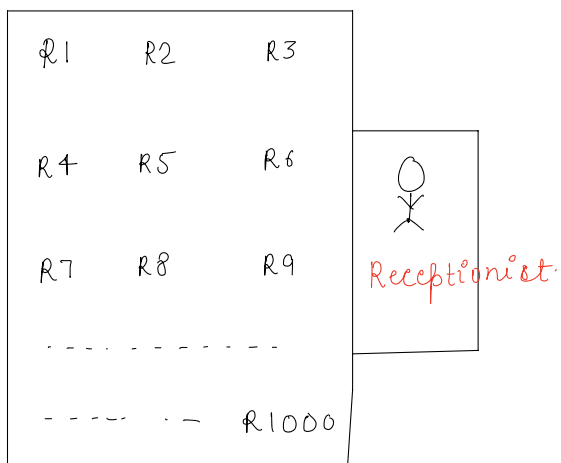
Hashmap



Solution:

Register

Room no	occupied
1	✓
2	✓
3	X
4	X
5	✓



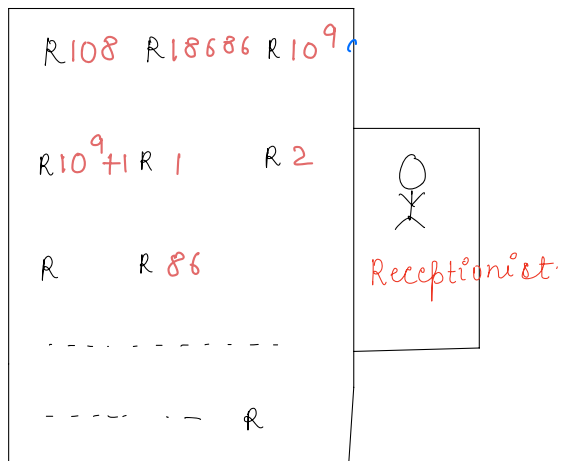
Register

Room no	occupied
1	✓
2	✓
3	X
4	X
5	X
...	...
1000	✓

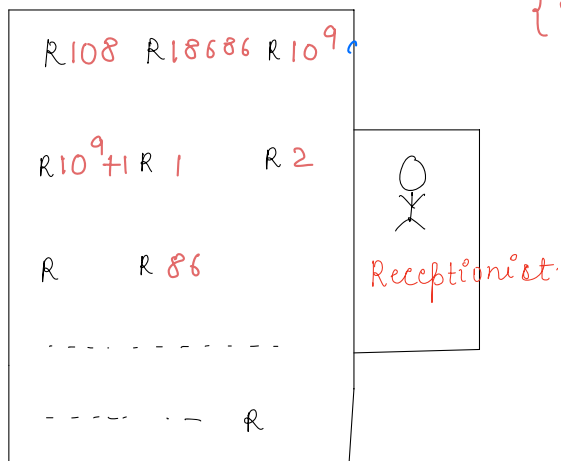
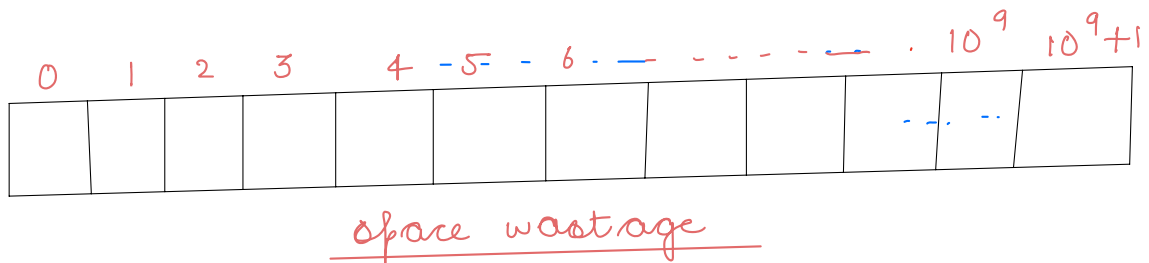
arr[]
↓
1001

0	1	2	3	4	5	6	...	1000
	t	t	f	t	f	t	...	f

Valid approach



boolean[]
occupied =
10⁹ + 2



Data structure

{ Room } no	Key	value { occupied }
	108	t
	18686	f
	10 ⁹	t
	10 ⁹ +1	f
	⋮	⋮
	⋮	⋮

Key, value ——— Map data structure

Map < Integer, Boolean > map;

1. searching ——— O(1) ——— future classes
2. Add ——— O(1)
3. Delete ——— O(1)
- ⋮

Quiz 1 which of the following hashmap will you use to store the population of every country?

Key ÷ Country name

value ÷ Population

Map < string, Long >

Quiz 2 which of the following hashmap will you use to store the number of states of every country?

Key ÷ Country name

value ÷ No of states

Map < string, Integer >

Quiz 3 which of the following hashmap will you use to store the name of all states of every country?

Key: Country name.

value: List<string>

Map<string, List<string>>

Quiz 4 which of the following hashmap will you use to store the population of each state in every country?

Key: Country name

value: population of each state

Map<string, Integer>

Map<string, Map<string, Integer>>

Key: Boolean.

{
key
true
false
}

Hashtet and hashmap functionalities

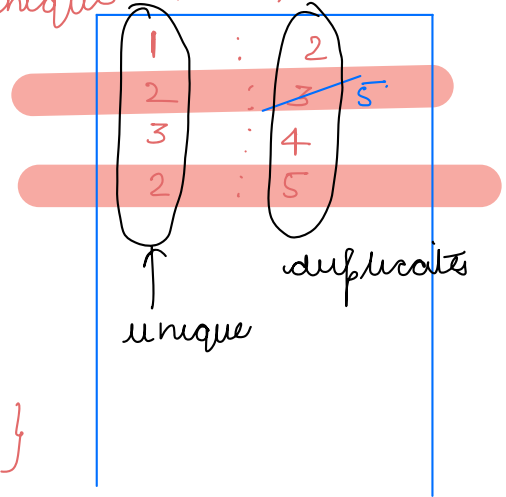
keys are unique $\langle \text{int}, \text{int} \rangle$

HashMap

1. > insert(key, value)

$O(1)$
1 : 2
2 : 3
3 : 4
2 : 5

java
key, value
.put()



2. > size() — 3 { no of key, value pairs }
 $O(1)$

3. > delete(key)

$O(1)$

delete(2)

delete(10)

exception

do nothing

4. > ~~update(key, value)~~ — insert

5. > search(key):
 $O(1)$

map.get(3) = 4

map.get(10) =

null

exception

Hash set

1. > insert(key, value)
O(1)

2. > size()
O(1)

3. > delete(key)
O(1) ..

4. > update(key, value)

5. > search(key) ..

Map — key, value
Set — key

Set <Integer>

→ keys of set are unique

Q. Given $arr[n]$ and Q queries. find frequency of the elements provided in a query.

ip

$A[] =$	2	6	3	8	2	8	2	3	8	10	6
---------	---	---	---	---	---	---	---	---	---	----	---

$Q = 4$

2	8	3	5
---	---	---	---

Element	frequency
2	3
8	3
3	2
5	0

Brute force:

```
for(i=0 _____ Q) {  
    el = queries[i]  
    cnt = 0  
    for(j=0 _____ n-1)  
        if (A[j] == el)  
            cnt++;  
    }  
    print(cnt);  
}
```

TC: $O(Q * n)$

SC: $O(1)$

$1 \leq Q \leq 10^5$

$1 \leq n \leq 10^5$

TC: $10^5 * 10^5 \approx 10^{10} \rightarrow \underline{\underline{TLE}}$

Idea 2:

A[] =

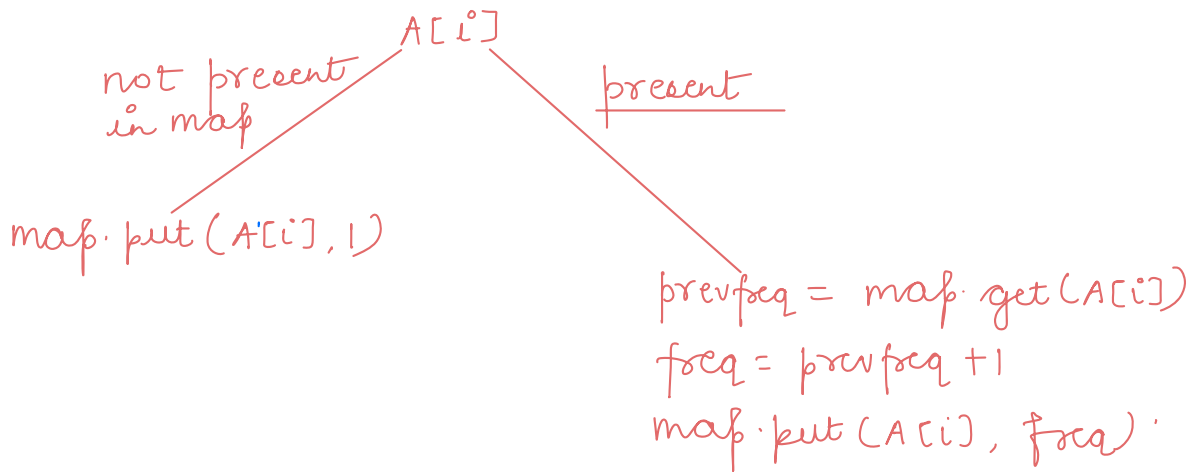
2	6	3	8	2	8	2	3	8	10	6
↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑

Map<Int, int>

<u>2</u>	:	1 2 3
6	:	1 2
3	:	1 2
8	:	(1) 2 3
10	:	1

Map<Int, Int>

2	:	3
6	:	2
3	:	2
8	:	2
10	:	1



Algorithm

```
void printfreq(int[] A, int[] Q) {
```

```
    map<int, int> map
```

```
    for(i=0 _____ n) { _____ n
```

```
        if( map.containsKey(A[i])) {
```

```
            freq = map.get(A[i])
```

```
            freq += 1;
```

```
            map.put(A[i], freq);
```

```
        } else {
```

```
            map.put(A[i], 1);
```

```
        }
```

```
    }
```

```
    for(i=0 _____ Q) { _____ Q
```

```
        curr = Q[i]
```

```
        if( map.containsKey(curr) {
```

```
            print( map.get(curr));
```

```
        } else {
```

```
            print( 0 );
```

```
        }
```

```
    }
```

```
}
```

TC: $Q + n$.

sc: $O(n)$

$1 \leq Q \leq 10^5$

$1 \leq n \leq 10^5$

$Q + n = 2 * 10^5$ — Accepted

Ques Given arr[n], find count of distinct elements.

3	5	6	5	4
---	---	---	---	---

ans $\Rightarrow$ 4

3	3	3
---	---	---

ans $\Rightarrow$ 1

1	1	1	2	2
---	---	---	---	---

ans $\Rightarrow$ 2

3	5	6	5	4
---	---	---	---	---

3	—	1
5	—	2
6	—	1
4	—	1

map.size() — Ans.

Set

3	5	6	5	4
---	---	---	---	---

< 3, 5, 6, 4 >
set.size() — Ans

Code

```
int countDistinctElements (int[] A) {  
    Set<Integer> set;  
    for (i = 0 _____ n) {  
        set.add(A[i]);  
    }  
    return set.size();  
}
```

TC: $O(n)$
SC: $O(n)$

Break: 10:17 - 10:27

Qu $A[n]$ and k , check whether there exists a pair (i, j) such that —

$$A[i] + A[j] = k \text{ and } i \neq j$$

0	1	2	3	4	5	6	7	8
8	9	1	-2	4	5	11	-6	4

$$k=6 \text{ true } \{arr[2] + arr[5]\}$$

$$k=22 \text{ false}$$

$$k=8 \text{ true } \{arr[4] + arr[8]\}$$

$$k=10 \text{ true } \{arr[1] + arr[2]\}$$

0	1	2	3	4	5
3	5	1	2	1	2

$$k=7 \text{ true } \{arr[1] + arr[3]\}$$

$$k=10 \text{ false } \{A[1] + A[1]\}$$

X

Brute force

2 loops

$$TC: O(n^2)$$

$$SC: O(1)$$

Expectation: $O(n)$ time

Approach 2

Wrong approach

0	1	2	3	4	5	6	7	8
8	9	2	-2	4	5	11	-6	4

, $k=9$.

target = ① ② 7 11
↑
return true

set

< 8, 9, 2, -2, 4, 5, 11, -6 >

Edge case:

0	1	2	3	4	5	6	7	8
8	9	2	-2	4	5	11	-6	4

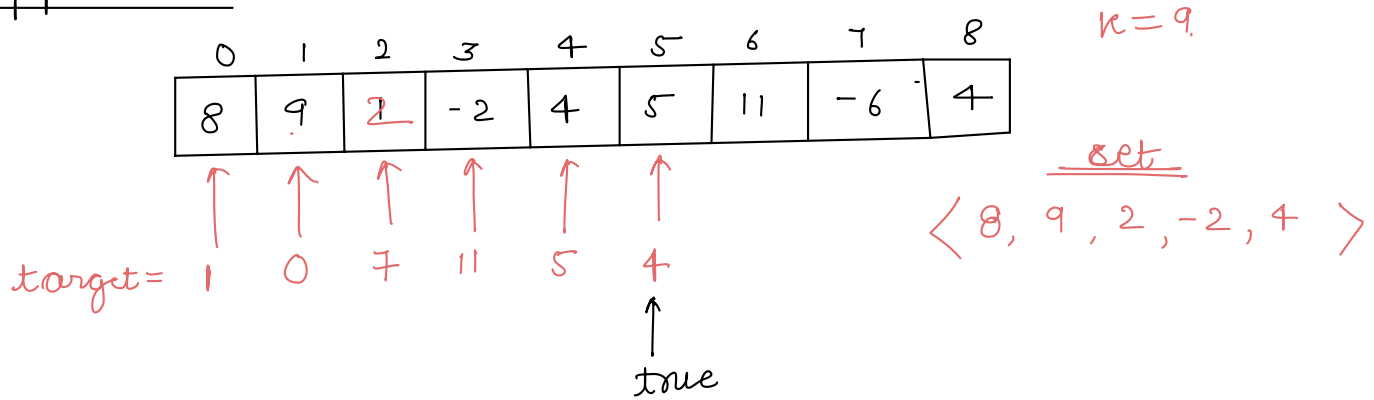
, $k=4$

target -4 -5 2
↑
true

< 8, 9, 2, -2, 4, 5, 11, -6 >

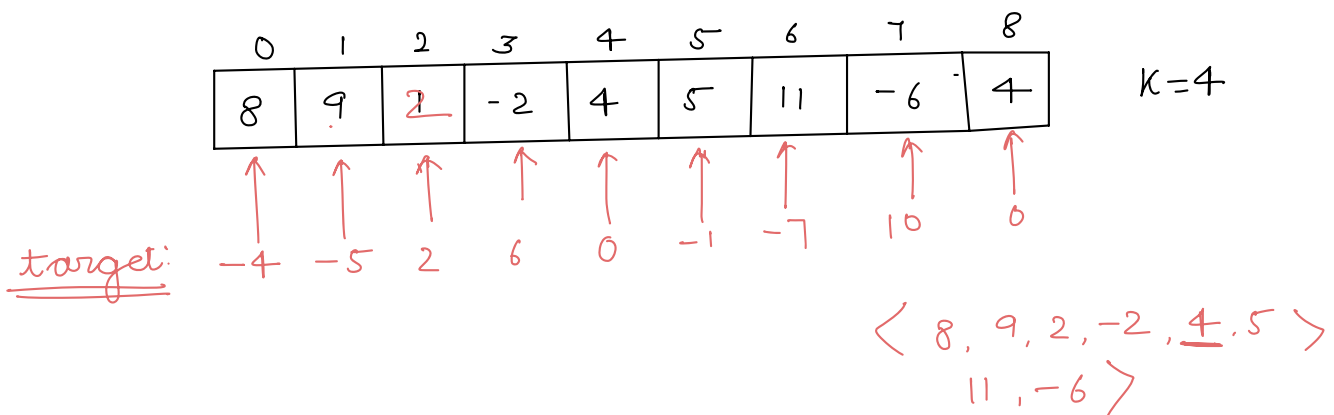
constraint : set

Approach 3



```
boolean targetsum(A[], k) {  
    n = A.length;  
    Set<Integer> set;  
    for (i = 0; i < n; i++) {  
        target = k - A[i];  
        if (set.contains(target)) {  
            return true;  
        }  
        set.add(A[i]);  
    }  
    return false;  
}
```

TC: $O(n)$
SC: $O(n)$



0	1	2	3	4	5
3	5	1	2	1	2

$K=3$

Count no of pairs ———

{2,3}

{2,5}

{3,4}

{4,5}

—————> $Ans = 4.$

0	1	2	3	4	5	6	7
2	5	2	5	8	5	2	8

$K=10$

target

↑ 8 ↑ 5 ↑ 8 ↑ 5 ↑ 2 ↑ 5 ↑ 8 ↑ 2

$(1) + (2) + (2) + (1) + (3)$ map < ^(e)int, ^(f)int>
 $Ans = 9$

2 : ~~1~~ ~~2~~ 3
5 : ~~1~~ ~~2~~ 3
8 : 1

function countPairs (A[], K) {

Map<int, int> map

ans = 0

for (i = 0 ————— n-1) {

target = K - A[i]

if (map.contains(target)) {

ans += map.get(target);

}

// insert A[i]

if (map.contains(A[i]) {

freq = map.get(A[i])

freq += 1;

map.put(A[i], freq);

} else {

map.put(A[i], 1);

}

}

return ans;

}

TC: $O(n)$
SC: $O(n)$

Thankyou 😊

Doubt

class which implements
HashMap map

(Interface)

Map

↑
{ key is
unique

unordered

1 : true	→ print	3 : false
2 : false		1 : true
4 : true		4 : true
3 : false		2 : false

Treemap

sorted keys

1 : true	→ print	1 : true
2 : false		2 : false
4 : true		3 : false
3 : false		4 : true

LinkedHashMap — ordering

1 : true	→ print	1 : true
2 : false		2 : false
4 : true		4 : true
3 : false		3 : false